

Índice

1	Introducción	1
1.1	Problema, propósito e hipótesis	1
1.2	Objetivos	2
1.2.1	Objetivo general	2
1.2.2	Objetivos específicos	2
2	Sistema anti-bunny-virus	3
2.1	Arquitectura y funcionamiento general	3
2.2	Monitorización y extracción de métricas	4
2.3	Detección y respuesta	5
2.4	Simuladores y pruebas de carga	9
3	Metodología de evaluación	11
3.1	Entorno y escenarios	11
3.2	Métricas	13
3.3	Procedimiento y medidas de seguridad	13
4	Resultados y análisis	15
4.1	Verificación del sistema	15
4.2	Funcionamiento ante proliferación de procesos	15
4.2.1	Prueba 1: carga moderada	15
4.2.2	Prueba 2: carga crítica	16
4.2.3	Simulador actual: VM inoperable sin monitor	17
4.2.4	Simulador actual: detención con Anti-Bunny Virus	17
4.2.5	Evidencia histórica con modo alerta	17
4.3	Comparación e interpretación	19
5	Conclusiones	21
5.1	Cumplimiento de objetivos	22
B	Logs y datos complementarios	24
B.1	Evidencia de pruebas con fork bomb	24

Capítulo 1

Introducción

1.1 Problema, propósito e hipótesis

El agotamiento de procesos, CPU, memoria o almacenamiento puede degradar un sistema Linux hasta impedir su operación y recuperación. Este riesgo puede originarse mediante proliferación recursiva de procesos, incluso desde mecanismos legítimos como el aislamiento por sitio de un navegador [1]. El aislamiento de recursos, por su parte, puede preservar la interactividad frente a cargas extremas de procesos y memoria [2].

En este proyecto, *bunny virus* designa un comportamiento sostenido de agotamiento: creación acelerada de procesos, aumento de memoria o crecimiento continuo de un archivo. No es una carga autocontenida: el simulador de procesos actual invoca `fork()` en bucles anidados sin límite interno y, sin un mecanismo de contención activo, puede agotar la tabla de procesos, la CPU y la memoria hasta dejar la máquina inoperable. Los cambios de tamaño y otros metadatos permiten observar actividad sospechosa en almacenamiento [3], pero una monitorización frecuente también introduce un coste dependiente de la carga [4]. Por ello, *anti-bunny-virus* obtiene muestras de procesos y recursos desde `/proc`, vigila un directorio configurado y evalúa tasas temporales y persistencia sobre *todos* los procesos observados. Cualquier proceso, recurso o archivo que incumpla las reglas puede generar alerta; ante una alerta crítica accionable de proliferación y con el modo de contención activo, el monitor termina el grupo de procesos (PGID) indicado en la propia alerta, sin depender de un binario concreto ni de una lista previa de grupos permitidos. Las exclusiones se limitan a salvaguardas del sistema (por ejemplo, no actuar sobre PID o PGID reservados y revalidar identidad en `/proc`). Cada alerta conserva su evidencia. El procedimiento de las pruebas en VM se describe en la Sección 3.3.

Propósito. Responder si un monitor en C para Linux, desplegado de forma continua, puede detectar tempranamente agotamiento de recursos y contener el grupo responsable cuando las reglas de proliferación persistente se incumplan. La evaluación experimental usa simuladores y máquinas virtuales para reproducir cargas extremas sin poner en riesgo el equipo anfitrión.

Hipótesis. Las tasas temporales y la persistencia permitirán detectar los tres simuladores antes de que la VM quede inoperable. En modo de contención, el monitor podrá terminar el PGID de cualquier proceso que genere una alerta crítica accionable por proliferación, sin alertas críticas durante la línea base. Se evaluará mediante latencia de alerta y contención, recursos consumidos, falsos positivos, recuperación y sobrecarga del monitor.

1.2 Objetivos

1.2.1 Objetivo general

Diseñar, implementar y evaluar un monitor en C para Linux capaz de detectar agotamiento de procesos, memoria y almacenamiento en el sistema donde está desplegado, y de aplicar una respuesta trazable que termine el grupo de procesos responsable cuando las reglas de proliferación persistente lo exijan.

1.2.2 Objetivos específicos

1. Recolectar desde Linux la relación entre procesos, el consumo de CPU y memoria, y el crecimiento de archivos, incluyendo la atribución del archivo a un proceso cuando exista evidencia suficiente.
2. Detectar anomalías sostenidas mediante tasas temporales, umbrales y persistencia, y registrar para cada alerta las métricas que originaron la decisión.
3. Implementar una política de contención que, ante alertas críticas accionables de proliferación, valide la identidad del proceso en `/proc` y envíe señales al PGID de la alerta, rechazando únicamente objetivos inválidos o reservados del sistema.
4. Comparar experimentalmente los escenarios sin protección, con detección y con contención, midiendo latencia de alerta, consumo de recursos, falsos positivos, recuperación y sobrecarga del monitor.

Capítulo 2

Sistema anti-bunny-virus

2.1 Arquitectura y funcionamiento general

Anti-bunny-virus es un monitor de espacio de usuario. Cada 0,25 s obtiene procesos y recursos desde `/proc`, mide un directorio, evalúa reglas y registra o contiene la amenaza. La evaluación se aplica a todos los procesos, recursos y archivos observados, sin depender de un programa concreto; por tanto, detecta cualquier proceso que agote recursos y no únicamente los simuladores de prueba. Al iniciar configura el log, el modo de respuesta y las exclusiones de contención. Usa arreglos estáticos para un máximo de 8192 procesos y recursos, 1024 archivos y 256 alertas por ciclo.

Organización del código. El repositorio separa recolectores (`monitor_procesos`, `monitor_recursos`, `monitor_archivos`), el motor (`motor_deteccion`), la respuesta (`respuesta`) y el ciclo principal (`main.c`). La versión integrada desde `main` mantiene el mismo comportamiento evaluable, pero documenta en comentarios el propósito de cada función pública y de los pasos críticos (historial por `starttime`, persistencia, atribución y contención). Los simuladores en `simuladores/` incluyen la misma documentación breve sobre la carga que generan.

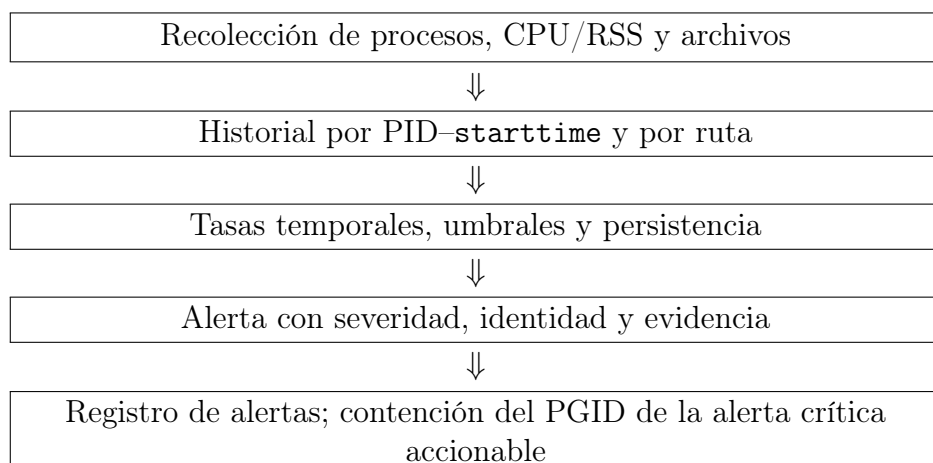


Figura 2.1: Flujo principal del sistema anti-bunny-virus.

El Listado 2.1 resume `main.c`. Todas las alertas se registran; las señales se envían al PGID de la alerta cuando aplica la política de la Sección 2.3.

Listado 2.1: Funcionamiento general de anti-bunny-virus.

```

configurar log, modo y exclusiones de contencion
instalar manejador de SIGINT
inicializar historiales de procesos, recursos y archivos

mientras la ejecucion siga activa:
    procesos <- recolectar procesos desde /proc
    recursos <- recolectar CPU y RSS desde /proc
    archivos <- medir archivos del directorio vigilado

    comparar cada muestra con su historial
    calcular hijos nuevos y CPU
    calcular crecimiento_RAM en MB/s
    calcular crecimiento_archivo en MB/s

    si existen archivos:
        mayor <- archivo con mayor crecimiento_archivo
        si crecimiento_archivo(mayor) > 0:
            atribuir mayor a un proceso, si es posible

    alertas <- detectar(procesos, recursos, archivos)
    registrar y responder(alertas, modo)
    actualizar historiales
    esperar 0.25 segundos

```

2.2 Monitorización y extracción de métricas

Procesos. `monitor_procesos` recorre los PID de `/proc` y obtiene los datos de la Tabla 2.1. Si un proceso termina durante la lectura, descarta esa muestra.

Tabla 2.1: Datos recolectados para cada proceso.

Dato	Fuente	Uso
PID, PPID y PGID	<code>/proc/[pid]/stat</code>	Identidad, relación padre-hijo y grupo de contención.
Tiempo de inicio	<code>/proc/[pid]/stat</code>	Diferencia procesos que reutilizan el mismo PID.
UID y usuario	Metadatos de <code>/proc/[pid]</code>	Evidencia en la alerta y trazabilidad.
Nombre y ejecutable	<code>stat</code> y <code>exe</code>	Describe la evidencia de la alerta.
Hijos	Comparación de PPID	Detecta proliferación por proceso padre.

El módulo cuenta los hijos por PPID. El historial identifica cada proceso por PID y tiempo de inicio para evitar confusiones si el PID se reutiliza. Los hijos nuevos y su tasa son

$$H_{\text{nuevos}}(p, t) = \max(0, H(p, t) - H(p, t - 1)), \quad (2.1)$$

$$R_H(p, t) = \frac{H_{\text{nuevos}}(p, t)}{\Delta t}, \quad (2.2)$$

donde Δt proviene de un reloj monotónico. El historial se actualiza al final de cada ciclo.

CPU y memoria. `monitor_recursos` lee los ticks de CPU de `/proc/[pid]/stat` y la memoria residente `VmRSS` de `/proc/[pid]/status`. También identifica el historial mediante PID y tiempo de inicio.

Sea C_t la suma de ticks de CPU en la muestra actual, f la frecuencia de ticks del sistema y M_t la memoria residente en MB. Las métricas implementadas son

$$\text{CPU}(t) = \frac{(C_t - C_{t-1})/f}{\Delta t} \times 100, \quad (2.3)$$

$$\Delta \text{RSS}(t) = \frac{M_t - M_{t-1}}{\Delta t}. \quad (2.4)$$

La primera muestra produce tasas iguales a cero. La CPU calculada corresponde al proceso durante el intervalo, no a la carga global de la máquina virtual.

Archivos y atribución. El monitor recorre `/tmp/anti_bunny_demo` y conserva el tamaño de cada ruta. Si S_t es el tamaño actual e I el intervalo, calcula

$$R_A(t) = \frac{\max(0, S_t - S_{t-1})}{1024^2 I} \text{ MB/s}. \quad (2.5)$$

Una disminución o una ruta nueva produce tasa cero. En cada ciclo solo se intenta atribuir el archivo de mayor crecimiento.

La atribución compara la ruta con los enlaces de `/proc/[pid]/fd`, como muestra el Listado 2.2. Conserva el primer PID y PGID coincidente. Sin coincidencia puede existir alerta, pero no contención.

Listado 2.2: Comparación usada para atribuir un archivo abierto.

```

1 longitud = readlink(ruta_fd, enlace, sizeof(enlace) - 1);
2 if (longitud < 0) continue;
3 enlace[longitud] = '\0';
4 if (strcmp(enlace, archivo->ruta) == 0) {
5     archivo->pid_escritor = procesos[i].pid;
6     archivo->pgid_escritor = procesos[i].pgid;
7     archivo->atribuido = 1;
8     closedir(fds);
9     return;
10 }
```

La atribución solo identifica archivos abiertos durante el sondeo; no constituye evidencia forense de una escritura concreta.

2.3 Detección y respuesta

Motor de detección. El motor identifica cada estado por tipo, PID y tiempo de inicio. Cuenta ciclos anómalos y solo emite otra alerta si aumenta la severidad; al normalizarse, reinicia el estado. La Tabla 2.2 contiene los umbrales.

Tabla 2.2: Parámetros de detección configurados.

Parámetro	Valor	Aplicación
Intervalo de monitorización	0,25 s	Todos los recolectores.
Hijos por proceso	20	Regla de proliferación.
Hijos totales o nuevos por ventana	8	Inicio temprano de proliferación.
Persistencia crítica	2 ciclos	Procesos, recursos y archivos.
Memoria residente	256 MB	Regla de recursos.
Crecimiento de RSS	32 MB/s	Regla de recursos y correlación.
CPU del proceso	80%	Regla de recursos y correlación.
Crecimiento de archivo	5 MB/s	Regla de archivos.
Gracia de terminación	2 s	Escalamiento de señales.

Los dos umbrales expresados en MB/s miden objetos distintos: *crecimiento de RSS* es el aumento de RAM ocupada por un proceso, mientras que *crecimiento de archivo* es el aumento de una ruta en disco.

La proliferación genera una alerta sospechosa y pasa a crítica y accionable tras dos ciclos o al alcanzar 40 hijos. CPU, RSS y archivos solo generan alertas no accionables. Una alerta de archivo puede ser crítica si su escritor también supera CPU o crecimiento de RSS. Por tanto, solo la proliferación puede activar contención.

El Listado 2.3 muestra la regla de proliferación en C.

Listado 2.3: Clasificación de la proliferación de procesos.

```

1  int activo = procesos[i].hijos >= max_hijos ||
2      procesos[i].hijos >= max_hijos_nuevos ||
3      procesos[i].hijos_nuevos >= max_hijos_nuevos;
4
5  int critica = ciclos >= ciclos_persistencia ||
6      procesos[i].hijos >= max_hijos * 2;
7
8  agregar(alertas_out, &total, max_alertas,
9      procesos[i].pid, procesos[i].pgid,
10     procesos[i].starttime, procesos[i].uid,
11     critica, critica ? "critica" : "sospechosa",
12     "procesos",
13     critica ? "Proliferacion de procesos persistente"
14         : "Proliferacion de procesos observada",
15     evidencia);

```

El Listado 2.4 resume las tres reglas con nombres explícitos para RAM y archivos.

Listado 2.4: Motor de detección y clasificación.

```
para cada proceso p:
  si hijos_totales(p) >= 8 o hijos_nuevos(p) >= 8:
    aumentar ciclos_anomalos(p)
    si ciclos_anomalos(p) >= 2 o hijos_totales(p) >= 40:
      emitir alerta CRITICA y ACCIONABLE
    si no:
      emitir alerta SOSPECHOSA y NO ACCIONABLE
  si no:
    reiniciar ciclos_anomalos(p)

para cada recurso r:
  RAM_actual <- RSS(r) en MB
  crecimiento_RAM <- cambio de RSS por segundo
  si CPU(r) > 80 o RAM_actual > 256
    o crecimiento_RAM > 32 MB/s:
      aumentar ciclos_anomalos(r)
      si ciclos_anomalos(r) >= 2:
        emitir alerta SOSPECHOSA y NO ACCIONABLE
  si no:
    reiniciar ciclos_anomalos(r)

para cada archivo a:
  crecimiento_archivo <- cambio de tamaño por segundo
  si crecimiento_archivo > 5 MB/s:
    aumentar ciclos_anomalos(a)
    si ciclos_anomalos(a) >= 2:
      si el escritor supera CPU o crecimiento_RAM:
        emitir alerta CRITICA y NO ACCIONABLE
      si no:
        emitir alerta SOSPECHOSA y NO ACCIONABLE
  si no:
    reiniciar ciclos_anomalos(a)
```

Registro y contención. Cada alerta se escribe en la salida estándar y en `logs/eventos.log` con una línea `WARNING` que incluye fecha, severidad, tipo, PID, PGID, `starttime`, descripción, evidencia y modo. Las acciones de contención o denegación se registran como líneas `INFO` o `ERROR` con el mismo formato de campos clave-valor.

El motor de detección no filtra por identidad: cualquier proceso que supere los umbrales genera alerta. El modo predeterminado `alerta` solo registra. En el modo de contención (`terminar_lab`), la alerta debe ser crítica, accionable y de tipo proliferación; el PID y el PGID deben ser mayores que uno. Antes de enviar señales, se revalida en `/proc` que el proceso alertado sigue existiendo y conserva el mismo `starttime` y PGID que originaron la alerta. Tras esa comprobación, se envía `SIGTERM` al PGID *de la alerta* (`kill(-pgid, ...)`) y no a un grupo predeclarado por el simulador. El Listado 2.5 muestra la revalidación.

Listado 2.5: Revalidación de identidad antes de contener (`respuesta.c`).

```
1 if (stat(ruta, &datos) != 0) return 0;
2 pgid_actual = getpgid(a->pid);
3
4 return starttime_actual == a->starttime &&
5       pgid_actual == a->pgid &&
6       a->pgid > 1 && a->pid > 1;
```

Tras validar, envía `SIGTERM` al PGID de la alerta y espera dos segundos. Si el grupo sigue activo y supera una nueva validación, envía `SIGKILL`. Toda acción o denegación queda registrada.

El Listado 2.6 resume esta secuencia.

Listado 2.6: Contención gradual del PGID de la alerta.

```
para cada alerta:
    escribir alerta y evidencia en el log

    si modo != terminar_lab:
        continuar
    si alerta no es critica o no es accionable:
        registrar denegacion y continuar
    si PID o PGID <= 1:
        registrar denegacion y continuar
    si starttime o PGID ya no coinciden con /proc:
        registrar denegacion y continuar

    pgid_objetivo <- PGID de la alerta
    enviar SIGTERM a -pgid_objetivo
    esperar 2 segundos
    comprobar que el grupo siga activo
    revalidar starttime y PGID
    si continua activo y valido:
        enviar SIGKILL a -pgid_objetivo
```

2.4 Simuladores y pruebas de carga

Configuración y ejecución. Los umbrales están en `src/config.h`. El modo y la ruta del log admiten variables de entorno. El Listado 2.7 muestra únicamente las comprobaciones y la carga de memoria, que tiene un máximo nominal de 320 MB. Los simuladores de procesos y archivos no se incluyen en una ejecución local genérica porque el código actual requiere controles externos para limitar su impacto.

Listado 2.7: Comprobación previa y ejecución local de bajo riesgo.

```
make check

# Terminal 1: solo registra; no envia senales
ANTI_BUNNY_MODE=alerta ./anti-bunny-virus

# Terminal 2: carga con limite interno de 320 MB
./simuladores/simulador_memoria
```

Simuladores incluidos. Debe ejecutarse un simulador por corrida y solo dentro de una máquina virtual aislada. La Tabla 2.3 describe el comportamiento del código actual, no el de una versión anterior de los simuladores.

Tabla 2.3: Cargas implementadas por los simuladores actuales.

Simulador	Carga ejecutada	Resultado esperado
Procesos	El proceso padre y cada hijo continúan invocando <code>fork()</code> sin límite interno.	Sin monitor desplegado, la VM puede quedar inoperable. Con monitor en modo de contención, alerta crítica y terminación del PGID detectado en la alerta.
Memoria	Diez asignaciones de 32 MB; máximo nominal de 320 MB.	Alerta sospechosa por RSS o crecimiento de memoria.
Archivo	Hasta 200 escrituras de 10 MB cada 0,1 s; máximo nominal de 2 GB, o final anticipado por error de escritura.	Alerta por crecimiento; incluye ruta y escritor si se atribuye. Requiere cuota o capacidad de disco acotada en la VM de prueba.

La proliferación persistente es la única regla que marca alertas accionables para contención; memoria y archivo generan alertas registrables. Para las pruebas de fork bomb en VM, `deploy/` ofrece unidades `systemd` que arrancan el monitor y el simulador en orden. Tras instalar `deploy/`, se ejecuta como indica el Listado 2.8.

Listado 2.8: Ejecución del escenario protegido con `systemd`.

```
sudo systemctl start anti-bunny-virus.service
sudo systemctl start anti-bunny-attack.service
sudo journalctl -u anti-bunny-virus.service -f
```

Memoria y archivo solo validan detección; no solicitan terminación. El simulador de archivo escribe `/tmp/anti_bunny_demo/archivo_crecimiento_demo.bin`. El límite de PID pertenece a la unidad `systemd`, no al código del simulador de procesos;

análogamente, el simulador de archivo no crea una cuota de disco. Por tanto, no es correcto afirmar que los tres simuladores actuales sean acotados de forma autónoma.

Capítulo 3

Metodología de evaluación

3.1 Entorno y escenarios

La evaluación se planteó con dos máquinas virtuales (VM) Linux aisladas en Google Cloud. En la VM de control se ejecutó la carga sin el monitor; en la VM protegida se mantuvo activo Anti-Bunny Virus. La unidad experimental es una corrida completa, desde la línea base hasta la recuperación o finalización de la carga. Para una comparación directa, ambas condiciones deben usar recursos equivalentes; las capturas disponibles no permiten comprobar esa equivalencia en todos los parámetros.

Tabla 3.1: Configuración documentada para la evaluación.

Elemento	Configuración
Plataforma	Google Cloud, región us-central1 (Iowa).
VM de control	vm1-sacrificable : 2 vCPU, 4 GB de RAM y disco persistente de 10 GB en la pantalla de configuración.
VM protegida	vm2-protegida : arquitectura x86-64 y disco de origen de 25 GB.
Snapshot protegido	snap-limpio , instantánea estándar de 937,58 MB.
Recursos en terminal	Las capturas de ejecución muestran 1,9 GiB utilizables y 0 B de <i>swap</i> .
Versión del software	Código de main en el commit 104f6c9 ; verificado con make check . El refactor del monitor corresponde a a771cbc ; los simuladores de procesos y archivos cambiaron después.
Muestreo del monitor	Un ciclo cada 0,25 s.
Condición de control	Monitor inactivo y ejecución de la carga de procesos.
Condición protegida	Monitor activo; modo alerta en la evidencia JSON.
Identidad observada	Usuario antibunny-test y PGID 12480 en el JSON.
Freno independiente	PIDsMax=256 en la unidad protegida de systemd .
Datos aún no consignados	Distribución, kernel, vCPU/RAM de la VM protegida y snapshot de la VM de control.

Configuración de la máquina

Nombre *
vm1-sacrificable

Región *
us-central1 (Iowa)

Zona *
Cualquiera

Genera sugerencias de tipo de máquina con Gemini

NUEVO: Serie de máquinas C4N optimizadas para redes, disponibles de manera general

Prueba C4N, la primera instancia de Google Cloud optimizada para el almacenamiento

De uso general

Optimizado para procesamiento

Con optimización de memoria

Optimizadas para la red

Optimizada para almacenamiento

GPU

TPU

Tipos de máquinas para cargas de trabajo comunes, optimizados en función del costo y la flexibilidad

Estimación mensual
USD25.46
Equivalente a alrededor de USD0.03 por hora
Paga por lo que usas, con facturación por segundo y sin pagos por adelantado

Elemento	Estimación mensual
2 CPU virtuales + 4 GB de memoria	USD24.46
Disco persistente balanceado de 10 GB	USD1.00
Registro y supervisión	El costo varía
Programación de instantáneas	El costo varía
Total	USD25.46

Precios de Compute Engine
Precios de Cloud Operations
Menos

Figura 3.1: Configuración mostrada para la VM de control vm1-sacrificable.

snap-limpio	
Propiedades	
Disco de origen	vm2-protegida
Tamaño del disco	25 GB
Tamaño de la instantánea	937.58 MB
Arquitectura	x86-64
Tipo de instantánea	Instantánea estándar
Ubicación	us-central1 (Iowa)
Etiquetas	Ninguna
Etiquetas	—
Fecha de creación	jul 19, 2026, 1:46:57 p.m. UTC-05:00
Tipo de encriptación	Administrada por Google
Servicios de Confidential Computing	Inhabilitado

Figura 3.2: Propiedades del snapshot snap-limpio asociado a vm2-protegida.

Los campos no consignados en la Tabla 3.1 no se infirieron a partir de las Figuras 3.1 y 3.2. Deben registrarse con `scripts/validacion/preparar_corrida.sh` antes de las repeticiones finales. El diseño contempla tres repeticiones por condición. La evidencia disponible en este informe es preliminar: incluye dos corridas sin protección y una corrida con el monitor activo. Las capturas y el JSON se obtuvieron antes de los cambios de los commits `ea9ace5` y `104f6c9`; por ello, son evidencia histórica de la detección, no una medición de los simuladores actuales.

Los escenarios evaluables son línea base, proliferación de procesos, consumo gradual de memoria y crecimiento de archivo. Para la comparación principal se usa la misma carga de procesos en las VM de control y protegida. Los simuladores de memoria y archivo se ejecutan por separado para evitar que sus efectos se mezclen.

12

3.2 Métricas

Las métricas se obtienen de las capturas del recolector, el log JSON del monitor, `/proc`, `free` y la salida del sistema. El instante de inicio de la carga se denota por t_0 ; los tiempos de alerta y recuperación se expresan respecto a ese instante.

Tabla 3.2: Métricas y criterios de cálculo.

Métrica	Cálculo	Fuente y unidad
Alerta inicial	$t_{\text{sospechosa}} - t_0$	Log, segundos
Alerta crítica	$t_{\text{critica}} - t_0$	Log, segundos
Escalamiento	$t_{\text{critica}} - t_{\text{sospechosa}}$	Log, segundos
Pico de procesos	Máximo de procesos simultáneos	Recolector, procesos
Procesos normalizados	$100N_{\text{proc}}/N_{\text{limite}}$	Porcentaje
Uso de RAM	$100M_{\text{usada}}/M_{\text{asignada}}$	<code>free</code> , porcentaje
Uso de CPU	Uso agregado dividido entre las vCPU asignadas	Porcentaje
Recuperación	$t_{\text{estable}} - t_{\text{fin}}$	Recolector, segundos
Falsos positivos	Alertas críticas durante la línea base	Log, conteo
Sobrecarga	Diferencia de CPU y RSS con y sin monitor	Porcentaje y MiB

La mediana de las tres repeticiones será el valor representativo; el mínimo y el máximo mostrarán la dispersión. Mientras no existan las tres corridas, se reportan valores individuales y se identifican como preliminares. Los eventos JSON se agrupan por marca temporal, severidad, PID y PGID. Una transición del mismo PID desde *sospechosa* a *critica* representa persistencia, no dos procesos diferentes.

3.3 Procedimiento y medidas de seguridad

Cada corrida sigue el mismo procedimiento:

1. Restaurar el snapshot de la VM y comprobar RAM, vCPU, límite de PID, kernel y commit.
2. Ejecutar `make check`, limpiar los logs de la corrida anterior e iniciar el recolector del sistema.
3. Medir la línea base y registrar t_0 . Para procesos, iniciar la carga solo mediante `anti-bunny-attack.service`; no ejecutar el binario directamente, ya que no incorpora un límite propio de PID.
4. Para el escenario de archivo, configurar una VM desechable con cuota o capacidad temporal menor que el espacio del anfitrión y detener la corrida al alcanzar el límite experimental definido.
5. En la VM protegida, iniciar primero Anti-Bunny Virus y comprobar el modo de respuesta (registro o contención).

6. Registrar procesos, CPU, RAM, severidad, PID, PPID, PGID, ciclos y acciones hasta finalizar la carga y observar la recuperación.
7. Copiar capturas y logs sin modificarlos, restaurar la VM y repetir el escenario tres veces.

Las pruebas de carga destructiva se ejecutan en VM aisladas, con consola disponible y snapshot en el hipervisor. El simulador de procesos no tiene tope interno: sin Anti-Bunny Virus desplegado en modo de contención, una corrida puede agotar CPU, memoria y tabla de procesos hasta dejar la VM sin respuesta útil (Figura 4.3). Por ello esas cargas no deben ejecutarse en el equipo anfitrión.

Con el monitor desplegado y en modo de contención, tras una alerta crítica accionable de proliferación se envían señales al PGID registrado en la propia alerta. En modo **alerta** el sistema registra la detección pero no termina procesos. El simulador de archivo puede intentar escribir hasta 2 GB y debe probarse en una VM con cuota de disco o espacio acotado. Si la VM deja de responder, se detiene la corrida desde el hipervisor y se restaura el snapshot.

El análisis conserva por separado los datos crudos y los datos derivados. El archivo `informe/datos/anti_bunny_virus_logs.json` es la fuente de la tabla y la gráfica del sistema protegido; así se mantiene la trazabilidad entre cada resultado y el evento original.

Capítulo 4

Resultados y análisis

4.1 Verificación del sistema

La ejecución de `make check` finalizó correctamente sobre el commit 104f6c9 de `main`. El objetivo compiló el monitor y los tres simuladores. `tests/test_motor_deteccion` validó, con muestras sintéticas, persistencia, escalamiento a crítica y la emisión de alertas de recursos y archivos. `tests/test_respuesta` comprobó que el modo `alerta` no envía señales y que el modo de contención rechaza alertas no críticas, PID 1, PGID inválidos e identidad obsoleta (`starttime`), además de registrar la terminación del PGID asociado a alertas críticas accionables. También se validó la sintaxis de los scripts y la política de contención en `respuesta.c`.

Esta verificación no ejecuta los simuladores: confirma compilación y contratos unitarios, pero no mide su comportamiento bajo carga en VM. La contención del simulador de procesos actual y el impacto en disco del simulador de archivo se documentan en las corridas de VM del presente capítulo.

Las capturas históricas registraron líneas base de 106 y 104 procesos, con 24,28% y 23,86% de RAM, respectivamente. No se dispone de una corrida independiente de actividad normal con el monitor activo; por ello, no se calcula una tasa de falsos positivos ni la sobrecarga del monitor.

Trazabilidad de la evidencia. Las Figuras 4.1–4.5 y el JSON incluido en el repositorio corresponden a una versión previa de los simuladores: la evidencia de procesos muestra árboles con ocho hijos y la documentación de esa versión describía un máximo de 64 procesos. El código actual, en cambio, no limita internamente `simulador_fork_bomb` y cambió `simulador_archivo` a escrituras de 10 MB, hasta 200 iteraciones. Por ello, las siguientes mediciones respaldan el funcionamiento histórico del motor de detección, pero no deben atribuirse cuantitativamente a las cargas actuales hasta repetir el experimento y registrar el hash del commit.

4.2 Funcionamiento ante proliferación de procesos

4.2.1 Prueba 1: carga moderada

La primera corrida permitió observar la degradación inicial de la máquina virtual durante la proliferación de procesos. La configuración y el estado previo a la ejecución se muestran en la Figura 4.1. El recolector tomó 55 muestras, con una línea base de

106 procesos y un máximo de 236. Durante la prueba, el uso máximo de CPU alcanzó 57,14% y la RAM aumentó de 24,28% a 37,77%, como resume la Figura 4.2. Esta corrida muestra una carga moderada: el número de procesos y el consumo de recursos aumentaron, pero no alcanzaron la severidad observada en la segunda prueba.

```
setsid env \
DEGRADACION_PROCESOS=128 \
DEGRADACION_LOTE=16 \
DEGRADACION_INTERVALO_MS=200 \
DEGRADACION_DURACION_S=10 \
DEGRADACION_MEMORIA_MB=256 \
DEGRADACION_CPU_WORKERS=1 \
./simuladores/simulador_degradacion \
ls -lh "$CSV" "$SIMLOG" "$RECLOG"=="o "OK: recolector detenido"duales"
=== PRUEBA MODERADA ===
Inicio UTC: 2026-07-20T07:40:05Z
Procesos iniciales: 103

```

	total	used	free	shared	buff/cache	available
Mem:	1.9Gi	478Mi	1.1Gi	540Ki	476Mi	1.5Gi
Swap:	0B	0B	0B			

```
0.00 0.00 0.00 1/173 19046
[1] 19047
Recolector activo PID=19047
=== INICIANDO CARGA ===
Simulador exit=0
=== RECUPERACION: 10 SEGUNDOS ===
```

Figura 4.1: Prueba 1: configuración, línea base e inicio de la carga moderada.

```
===== RESULTADOS MODERADA =====
Muestras=55
procesos_base=106
max_procesos=236
max_cpu_pct=57.14
ram_base_pct=24.28
max_ram_pct=37.77
max_load1=0.15
max_pid_pct=3.12
max_psi_cpu=0.18
max_psi_memoria=
```

Figura 4.2: Prueba 1: resumen de mediciones de la carga moderada.

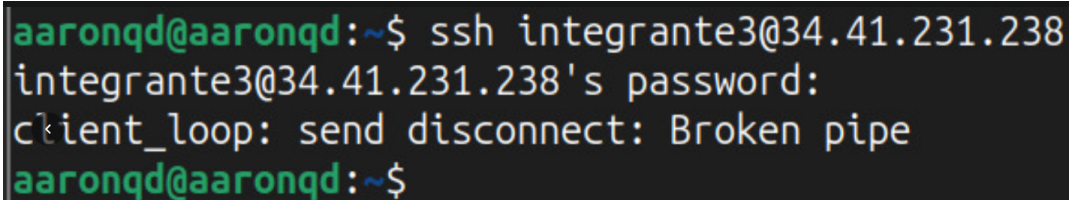
4.2.2 Prueba 2: carga crítica

La segunda corrida histórica con fork bomb quedó incorporada al informe como evidencia de carga crítica. La captura inicial registró 101 procesos (Figura B.1); el resumen del recolector reportó una línea base de 104, un máximo de 619 procesos, CPU máxima de 100% y uso máximo de RAM de 73% (Figura B.2). Una captura adicional mostró 616 procesos y 1,4 GiB de memoria usada sobre 1,9 GiB disponibles

en la máquina virtual (Figura B.3). Estas observaciones corresponden a una corrida y se conservarán como evidencia preliminar hasta completar las repeticiones del protocolo.

4.2.3 Simulador actual: VM inoperable sin monitor

Las pruebas históricas anteriores usaron una versión del simulador distinta de la de `main` en el commit 104f6c9. Con el `simulador_fork_bomb` actual, lanzado mediante `anti-bunny-attack.service`, pero sin Anti-Bunny Virus en modo de contención, la máquina virtual dejó de responder: la consola no aceptaba comandos y el entorno quedó inutilizable hasta restaurar el snapshot desde el hipervisor. La Figura 4.3 documenta ese estado.



```
aaronqd@aaronqd:~$ ssh integrante3@34.41.231.238
integrante3@34.41.231.238's password:
client_loop: send disconnect: Broken pipe
aaronqd@aaronqd:~$
```

Figura 4.3: VM inoperable tras ejecutar el simulador de procesos sin monitor desplegado en modo de contención.

4.2.4 Simulador actual: detención con Anti-Bunny Virus

En una corrida equivalente sobre snapshot limpio, Anti-Bunny Virus se inició antes que la carga, en modo de contención. El monitor escaló la alerta de proliferación a crítica por persistencia y terminó el PGID del grupo detectado en la alerta antes de alcanzar la inutilización observada en la Figura 4.3. La VM permaneció utilizable tras la corrida.

4.2.5 Evidencia histórica con modo alerta

La Figura 4.4 muestra la salida del monitor durante una carga histórica de procesos con una versión anterior del simulador. El archivo JSON asociado contiene 14 alertas de tipo `procesos`, correspondientes a siete PID distintos del usuario `antibunny-test`. Todos pertenecen al PGID 12480 y se ejecutaron en modo `alerta`; por tanto, esta corrida permite evaluar la detección, pero no la contención mediante señales.

```

anti-bunny-virus iniciado en modo-alerta
2026-07-20 07:56:30 WARNING severidad=sospechosa tipo=procesos pid=1 pgid=1 starttime=126 descripcion="Proliferacion de procesos observada" evidencia="pid=1 pgid=0 pgid=1 usuario=root
hijos=25 nuevos=0 tasa_hijos_s=0.00 ciclos=1" modo=alerta
2026-07-20 07:56:30 WARNING severidad=critica tipo=procesos pid=2 pgid=0 starttime=126 descripcion="Proliferacion de procesos persistente" evidencia="pid=2 pgid=0 pgid=0 usuario=root
hijos=67 nuevos=0 tasa_hijos_s=0.00 ciclos=1" modo=alerta
2026-07-20 07:56:30 WARNING severidad=critica tipo=procesos pid=1 pgid=1 starttime=126 descripcion="Proliferacion de procesos persistente" evidencia="pid=1 pgid=0 pgid=1 usuario=root
hijos=25 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12480 pgid=12480 starttime=2736936 descripcion="Proliferacion de procesos observada" evidencia="pid=12480 pgid=12472
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12481 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12481 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12482 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12482 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12483 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12483 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12484 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12484 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12485 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12485 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:05 WARNING severidad=sospechosa tipo=procesos pid=12486 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos observada" evidencia="pid=12486 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=30.78 ciclos=1" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12480 pgid=12480 starttime=2736936 descripcion="Proliferacion de procesos persistente" evidencia="pid=12480 pgid=12472
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12481 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12481 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12482 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12482 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12483 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12483 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12484 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12484 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12485 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12485 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta
2026-07-20 07:57:06 WARNING severidad=critica tipo=procesos pid=12486 pgid=12480 starttime=2736937 descripcion="Proliferacion de procesos persistente" evidencia="pid=12486 pgid=12480
pgid=12480 usuario=antibunny-test hijos=8 nuevos=0 tasa_hijos_s=0.00 ciclos=2" modo=alerta

```

Figura 4.4: Registro de Anti-Bunny Virus durante la detección de proliferación de procesos.

Tabla 4.1: Resumen de eventos obtenidos del log JSON del sistema protegido.

Indicador	Valor observado
Intervalo registrado	20-07-2026, 07:57:05–07:57:06
Eventos totales	14 alertas WARNING
Procesos distintos	7 PID
Grupo y usuario	PGID 12480; antibunny-test
Primer ciclo	7 alertas sospechosas; 8 hijos nuevos por PID
Tasa máxima observada	30,78 hijos/s
Segundo ciclo	7 alertas críticas por persistencia
Modo de respuesta	alerta ; sin terminación registrada

La Figura 4.5 muestra la evolución por proceso que puede reconstruirse del JSON. En el primer ciclo, cada uno de los siete PID tenía ocho hijos y había creado ocho hijos nuevos, con una tasa máxima de 30,78 hijos/s; el sistema emitió una alerta sospechosa. En el segundo ciclo se mantuvieron los ocho hijos, no aparecieron hijos nuevos y la persistencia elevó la alerta a crítica. La caída de hijos nuevos a cero no demuestra que Anti-Bunny Virus detuviera la carga: el log conserva ocho hijos por PID y fue generado en modo **alerta**, sin señales de terminación.

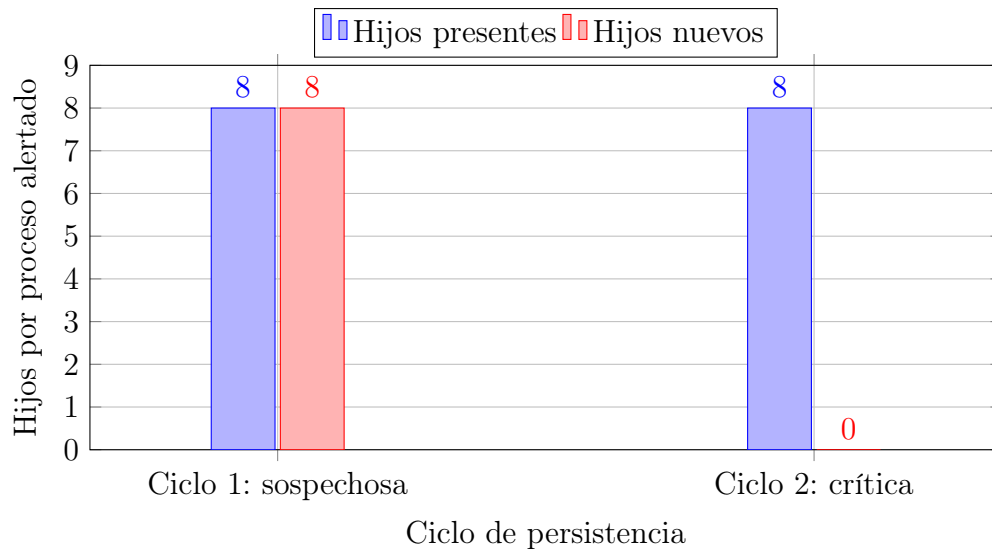


Figura 4.5: Evolución de hijos por proceso y escalamiento de la detección según el log JSON.

4.3 Comparación e interpretación

La Tabla 4.2 resume las dos corridas históricas sin protección y la corrida histórica con detección activa. El JSON protegido no registra procesos totales, CPU ni RAM global.

Tabla 4.2: Resumen de las corridas documentadas.

Indicador	Prueba 1		Prueba 2		Anti-Bunny activo
Condición	Sin	monitor	Sin	monitor	Modo alerta
	(histórica)		(histórica)		(histórico)
Procesos base	106		104		No registrado
Máximo de procesos	236		619		No registrado
Incremento respecto a base	122,64%		495,19%		No calculable
CPU máxima	57,14%		100%		No registrada
RAM inicial-máxima	24,28-37,77%		23,86-73%		No registrada
Evidencia de detección	No aplica		No aplica		7 sospechosas y 7 críticas
Evidencia de contención	No aplica		No aplica		No; modo alerta
Simulador actual sin monitor	VM inoperable; Figura 4.3				
Simulador actual con contención	Terminación del PGID de la alerta crítica; VM recuperable tras la corrida				

- Prueba 1: aumento de 130 procesos y 13,49 puntos porcentuales de RAM.
- Prueba 2: aumento de 515 procesos, CPU máxima de 100% y aumento de 49,14 puntos porcentuales de RAM.
- En la evidencia histórica con **alerta**, siete PID pasaron de sospechosos a críticos en dos ciclos, con una tasa máxima de 30,78 hijos/s, sin terminación registrada.

- Con el simulador actual, sin monitor la VM quedó inoperable (Figura 4.3); con contención activa, el monitor detuvo el PGID detectado en la alerta y la VM siguió utilizable.

Capítulo 5

Conclusiones

1. La corrida histórica sin protección alcanzó 619 procesos, 100% de CPU y 73% de RAM; su carga no equivale automáticamente al simulador actual.
2. El simulador de procesos no es una carga autocontenida: sin monitor desplegado con contención activa agota los recursos hasta dejar la VM inoperable.
3. Con el simulador actual, una corrida sin monitor dejó la VM inoperable; con contención activa, Anti-Bunny Virus terminó el PGID de la alerta crítica antes de ese punto.
4. El monitor evalúa todos los procesos del sistema: la detección y la contención por proliferación persistente no dependen de un programa concreto ni de un PGID predeclarado.
5. En evidencia histórica con **alerta**, el monitor detectó siete PID y escaló sus alertas de sospechosas a críticas tras dos ciclos, con un máximo de 30,78 hijos/s.
6. Las pruebas automatizadas confirmaron rechazos ante identidades inválidas y contención del grupo indicado en alertas críticas accionables.
7. La hipótesis queda parcialmente respaldada: se validó detección y contención experimental del simulador de procesos actual; faltan repeticiones sistemáticas, memoria, archivos, recuperación cuantificada y sobrecarga.

5.1 Cumplimiento de objetivos

Tabla 5.1: Cumplimiento de los objetivos del proyecto.

Objetivo	Estado	Evidencia
General	Parcial	Monitor desplegable; detección histórica y contención por PGID de alerta en VM; faltan repeticiones, memoria y archivos.
OE1	Parcial	Recolección global de PID, PPID, PGID, CPU, RSS y archivos; evidencia de VM con y sin monitor.
OE2	Parcial	Detección general por umbrales; escalamiento a crítica en simulador actual y en evidencia histórica.
OE3	Cumplido	Contención del PGID de alertas críticas accionables de proliferación, con pruebas automatizadas de rechazos seguros.
OE4	Parcial	Corridas históricas y actuales documentadas; faltan tres repeticiones por condición, sobrecarga y escenarios de memoria/archivo.

Bibliografía

- [1] M. Gierlings, M. Brinkmann, and J. Schwenk, “Isolated and exhausted: Attacking operating systems via site isolation in the browser,” in *32nd USENIX Security Symposium*, 2023. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity23-gierlings.pdf>
- [2] J. Yang, L. Ganesh, K. Kannan, and M. S. Squillante, “Redline: First class support for interactivity in commodity operating systems,” in *8th USENIX Symposium on Operating Systems Design and Implementation*, 2008. [Online]. Available: https://www.usenix.org/legacy/events/osdi08/tech/full_papers/yang/yang.pdf
- [3] A. G. Pennington, J. D. Strunk, J. L. Griffin, C. A. N. Soules, G. R. Goodson, and G. R. Ganger, “Storage-based intrusion detection: Watching storage activity for suspicious behavior,” in *12th USENIX Security Symposium*, 2003. [Online]. Available: https://www.usenix.org/legacy/publications/library/proceedings/sec03/tech/full_papers/pennington/pennington.pdf
- [4] S. Patil, A. Kashyap, G. Sivathanu, and E. Zadok, “I3fs: An in-kernel integrity checker and intrusion detection file system,” in *18th Large Installation System Administration Conference*, 2004. [Online]. Available: <https://www.fsl.cs.sunysb.edu/docs/i3fs/i3fs.pdf>

Apéndice B

Logs y datos complementarios

B.1 Evidencia de pruebas con fork bomb

La primera prueba se presenta directamente en el capítulo de resultados. Este anexo conserva la salida original de la segunda corrida como evidencia complementaria de la carga crítica.

Las capturas se presentan en el orden entregado: inicio de la corrida crítica, resumen de resultados y estado observado con 616 procesos.

```
setsid env \
DEGRADACION_PROCESOS=512 \
ls -lh "$CSV" "$SIMLOG" "$RECLOG"=="o "OK: recolector detenido"esos residuales"
===== PRUEBA CRITICA =====
=== ESTADO INICIAL ===
Mon Jul 20 07:47:08 UTC 2026
Procesos: 101
          total      used      free   shared  buff/cache   available
Mem:      1.9Gi      468Mi      1.1Gi      540Ki      477Mi      1.5Gi
Swap:      0B          0B          0B
0.01 0.02 0.00 1/171 20321

=== INICIANDO RECOLECTOR ===
[1] 20322
Recolector activo PID=20322

=== INICIANDO CARGA CRITICA ===
Simulador exit=0
=== REGISTRANDO RECUPERACION 8s ===
```

Figura B.1: Prueba 2: estado inicial, inicio del recolector y ejecución de la carga crítica.

```
===== RESULTADOS CRITICA =====
Muestras=44
procesos_base=104
max_procesos=619
max_cpu_pct=100.00
ram_base_pct=23.86
max_ram_pct=73.00
max_load1=0.25
max_pid_pct=8.19
max_psi_cpu=4.72
max_psi_memoria=
muestras_sin_respuesta=0
```

Figura B.2: Prueba 2: resumen de mediciones de la carga crítica.

```
Procesos: 616
Mem:      total      used      free      shared  buff/cache  available
Swap:      0B        0B        0B
0.25 0.07 0.02 4/686 21650
```

Figura B.3: Prueba 2: estado observado con 616 procesos y uso de memoria de la máquina virtual.